Jonghwan Shin

DevOps, Computational Biology

Tech Stack

- **Main** : DevOps · Distributed Systems · Workflow Automation · Computational Biology
- **Sub** : Cloud(AWS) · Backend · Full-stack · DBA · AI
- **Skills (main) :** Python · MySQL · Linux · Shell · Django · Javascript · Docker · Conda · Git · Gitlab · Snakemake · Cron · RDB (MySQL) · Prometheus · Grafana · Nginx (Ingress) · SGE (Sun Grid Engine) · REST API / HTTP API Integration · NGS analysis (Demultiplexing, WGS, WES, Single cell, TCR-seq, Illumina ICA etc)
- **Skills (sub) :** C · C++ · Go · Vite + React · TypeScript · AWS · Proxmox · IaC (Ansible) · Kubernetes · OpenVPN · CI/CD (Git, Jenkins) · Nextflow · Redis · RabbitMQ · NFS · Web DB (IndexedDB, Session Storage) · Security (Encryption, MFA, TLS)

Personal Information

Name	Jonghwan Shin		
Email	shinejh0528@gmail.com		
Links	Linkedin	GitHub	DockerHub
Portfolio	English (PDF) · Korean (PDF)		

Education

Degree	Status	University	Major	GPA	Period
M.S.	Withdrew	Yonsei University	Systems Biology	3.8/4.3	2019. 03 – 2019. 08
B.S.	Graduated	Kongju National University	Life Science	3.97/4.5	2013. 02 – 2019. 02

Certifications

Certification	Issuer	Credential ID	Date
AWS Certified SysOps Administrator - Associate	AWS	502211684	2025. 04
AWS Certified Developer – Associate	AWS	499361756	2025. 03
AWS Certified Solutions Architect – Associate (Expired)	AWS	446818627	2023. 05

Military Service

Branch	Status	Period
Republic of Korea Army	Sergeant, Honorable Discharge	2014. 12 – 2016. 09

Project Experience

DevOps, Computational Biologist **Theragenbio** **2022. 02 – Current**

- **(Project Lead, 2023. 05 – Present) Analysis Automation Web Service: Design, Development, Operations & Infrastructure Modernization**

✓ Tech Stack : Python, Django, MySQL, Docker, Kubernetes, Ansible, Nginx(Ingress), Prometheus, Grafana, Snakemake, SGE, Linux, Cron, NFS, Distributed Computing

✓ Key Features

- Led end-to-end design, development, and operation of the analysis automation web service, including backend, frontend, and database
- Migrated services to a 3-node Proxmox VM cluster and automated initial provisioning and maintenance with IaC (Ansible)
- Migrated docker-compose based operations to Kubernetes and switched to domain routing via an Ingress Controller
- Upgraded MySQL master-slave replication to a MySQL InnoDB Cluster (Router/Primary/Replica) on the Kubernetes MySQL

Operator

- Standardized shared storage with an NFS provisioner; applied monitoring, alerting, and resource-limit policies with K8s Dashboard, Prometheus, and Grafana

- Operated an SGE-based cluster of 20+ servers (192 threads / 1 TB memory) and automated the data lifecycle (deletion/migration) at 10 PB scale

✓ Result : Achieved fully automated, hands-off processing for 80% of all in-house analyses

• **(Sole Developer, 2026. 09 – 2026. 11) Designed and developed bamqc_mem, an integrated WGS QC tool**

✓ Tech Stack : C++, HTSlib, Multithreading, SIMD(AVX2), Python(matplotlib), Docker, Git

✓ Key Features

- Produces the outputs of 7 existing QC tools (2 bedtools, 3 Picard, samtools stats, Qualimap) from a single pass over BAM/CRAM; outputs are byte-for-byte identical to the original tools

- Index-based parallel processing over coordinate ranges (split by compressed bytes) plus replay of order-dependent statistics guarantees identical results for parallel and single-threaded runs

- Streaming design keeps memory usage independent of input data size

- CPU-level optimization: SIMD (AVX2) bitmask operations (auto-selected at runtime based on CPU support), branchless code, elimination of redundant computation, and library acceleration (libdeflate) → an additional 1.7x speedup over the previous version with identical output

✓ Result : Best-in-class performance among publicly available tools worldwide

- Fastest with the lowest memory at every thread count among all tools compared, including the latest PanDepth (2024) and mosdepth (coverage, 32 threads: up to 2.0x faster than PanDepth with 1/11 the memory; up to 31x faster than mosdepth)

- Reduced the time to produce all 7 QC outputs by 73x vs. the previous parallel approach (8,043 s → 110 s), with 1/17 the CPU and 1/43 the memory

- 76x faster even when processing 8 samples concurrently; memory 1,770 GB → 40 GB

• **(Sole Developer, 2026. 07 – 2026. 11) Designed and developed GENOA, a unified human / nonhuman WGS analysis pipeline**

✓ Tech Stack : Snakemake, Python, R(RMarkdown), GATK, BWA, snpEff, SGE, Singularity, Docker, Git

✓ Key Features

- human / nonhuman WGS unified into a single pipeline (species-specific references separated into config; alignment -> BQSR -> variant calling -> annotation -> single HTML report)

- Optimized parallel analysis with data sharding: variant calling is split evenly by contig intervals, removing the bottleneck where the largest chromosome (chr1) fixed the critical path

- Applied in-house optimized tools

· bamqc_mem (C++) : Replaces 7 existing QC tools with a single BAM pass; 45x faster QC with 1/40 the memory

· combine_gvcf_mem (C++) : GATK CombineGVCFs (Java): fixed its excessive memory usage; 93x less memory (3.07 GB -> 34 MB) and 3.7x faster

✓ Results

- human WGS, 30X, 20 samples: full analysis completed in 11 hours (benchmark)

- Normalized to 20 samples in parallel: 1.5x faster than Illumina's published standard BWA-MEM + GATK, 3.7x faster than Broad Terra CPU-based GATK, 4.6x faster than NVIDIA's published CPU-based GATK, and 2x faster than the previous in-house pipeline measured on the same cluster

• **(Sole Developer, 2025. 11 – 2026. 06) Designed and developed the reference data / build management system for the analysis platform (TGF)**

✓ Tech Stack : Python, Django, MySQL, Singularity/Docker, SGE, Git

✓ Key Features

- Replaced hard-coded, manually managed references in each pipeline with centralized DB-based management (4 layers: assembly -> geneset/BED -> build tool -> pipeline config)

- Automated source download by NCBI/Ensembl standard identifiers (assembly, accession), pre-build format validation, and per-tool index builds

- Registered build tools (bwa, STAR, RSEM, snpEff, bismark, cellranger, etc.) by tool/version; build outputs auto-generate per-pipeline reference configs linked to the analysis submission UI options

- Besides new builds, provided custom registration so users can register their own references directly on the web

✓ Results

- Centralized and automated reference data / build management for each analysis pipeline

- Registered in production: 128 assemblies, 539 genesets, 578 builds, 127 pipeline configs

• **(Team Member, 2025. 10 – 2026. 02) 10x Genomics OCM FASTQ demultiplexing pipeline/workflow development and operations automation**

✓ Tech Stack : Python, C++, Docker, Snakemake, Conda, MySQL, Cron, Django, API integration (HTTP/REST)

✓ Key Features

- End-to-end automation from demultiplexing → analysis job creation/execution/status updates, integrated with LIMS and the in-house TGF analysis automation web service (Django)

- Implemented LIMS API integration that periodically polls (GET) for newly registered samples and automatically checks analysis completion to update required values (PATCH)

- Built the analysis execution flow as a Snakemake workflow and standardized runtime environments with Conda/Docker for reproducibility and easy deployment

- Automated job flow with Cron scheduling and retry/re-run logic; improved failure-point identification and operational stability

- Developed the demultiplexing compute core in C++ with options for thread count and number of samples processed in parallel; achieved high throughput on large FASTQ files through chunked processing and parallelization

- Automatically recorded demultiplexing results and metadata in MySQL (LIMS), providing history/traceability and a basis for operational reporting

✓ Results

- Achieved demultiplexing throughput of ~4.3 GB per minute (varies with data/environment)

- Automated the entire workflow via LIMS + analysis automation web service integration (sample registration check → analysis execution → completion verification → result/status updates), greatly reducing manual work

• **(Team Member, 2025. 02 – 2026. 06) Commercial web service for a patented in-house deep learning analysis pipeline**

✓ Tech Stack : Python, Linux, Conda, Vite+React, PostgreSQL, RabbitMQ, Docker, Go, Java

✓ Key Features

- Owned Python backend development (core service features: authentication, tokens, logging, usage tracking)

- Distributed modules to users via CLI while running the actual analysis only on in-house servers through APIs, ensuring security and commercial viability

- Improved analysis request throughput with RabbitMQ-based parallel processing and multi-server distribution

- Provided pip installation and install/uninstall scripts, Docker support, and automatic post-processing after analysis completion

✓ Result : Technology patent registered; listed as an author on a related paper (submitted, under review)

• **(Project Lead, 2025. 08 – 2025. 10) Illumina ICA human WGS delivery automation pipeline**

✓ Tech Stack : Python, Docker, Snakemake, ICA API(icav2), Bioinformatics tools

✓ Key Features

- Automated project creation, analysis job submission, and monitoring via the ICA API
- Implemented end-to-end post-processing including data download, VCF annotation, and automatic generation of the delivery folder structure
- Automated ICA project cleanup and data deletion for cost management

✓ Result : Automated delivery and reduced operational burden

• **(Team Member, 2024. 11 – 2025. 12) Sensitive data protection + NGS demultiplexing automation for a national bio big data project**

✓ Tech Stack : Python, Snakemake, Linux, Cron, MySQL, AES Encryption, YAML Config

✓ Key Features

- Developed a preprocessing automation system meeting sensitive data protection requirements in a government project environment
- Managed operational parameters with YAML config, designed to adapt flexibly to changing environments and requirements
- Structured in stages: sequencing data detection → demultiplexing job detection → demultiplexing pipeline execution
- Designed the workflow to automatically merge with previous production history, if any, before delivery

✓ Result : Built an automated operation that meets sensitive data protection requirements

• **(Project Lead, 2024. 07 – 2024. 12 (phase 1) / 2026.06 – 2026. 07 (phase 2)) Jenkins-based CI/CD pipeline integrated with GitLab: design and development**

✓ Tech Stack: Jenkins(JCasC), GitLab Webhook, Kubernetes/Docker, Python, Git, Linux, YAML, pytest, ruff

✓ Key Features

- Standardized and centrally managed projects (repos), jobs, branches, and connection info (GitLab/host SSH/tokens) with YAML (config.yaml) + Jenkinsfile, designed to scale to multiple projects
- Jenkins jobs call in-house Python/Shell tools for deployment standardization — VERSION-based automatic rc tagging, production worktree update deployment, idempotent webhook registration, and LLM-generated release notes from git diff
  · On staging push, GitLab Webhooks (push events, branch filters) automatically trigger the pipeline: test → rc tag → deployment approval gate → production release + release tag
  · Deployment approval mode (automatic / manual approval in the Jenkins web UI) controlled by auto_approve in config.yaml
  · Builds run on Kubernetes dynamic agents (a pod per build); Jenkins configuration and credentials managed as code with JCasC
- Receives branch/merge/tag events via GitLab Webhooks to automatically trigger stage-specific pipelines
- Selects the functions to run in each pipeline job using the Active Choices Parameter plugin or YAML

✓ Result: Standardized deployment procedures across dev → staging → production branch flows and improved operational stability with approval gates and automatic tagging

• **(Project Lead, 2023. 05 – 2024. 07) Integrated output validation system (algorithm) for analysis pipelines**

✓ Tech Stack : Python, MySQL

✓ Key Features

- Designed an algorithm and system to centrally manage output validation for analysis pipelines developed in various languages and custom approaches
- Implemented rules defined by combining parameters, functions, and Python syntax to absorb variations in output formats across pipelines
- Classified output types as sample (multiple, one per sample) / single (one overall) and standardized the validation logic
- Designed as an extensible sub-project of the analysis automation web service, accounting for various exception cases in

operation

✓ Result : Standardized per-pipeline output validation criteria, improving the consistency of operational checks and maintenance efficiency

- **(Project Lead, 2023. 11 – 2024. 05) NGS demultiplexing (raw data preprocessing) end-to-end automation**

✓ Tech Stack : Python, MySQL, Django, Linux, Cron, distributed computing

✓ Key Features

- Led the raw data preprocessing (demultiplexing) process from design to implementation, automating the entire flow end-to-end
- Reliably automated the highly complex single-cell demultiplexing process from an operations perspective

✓ Result : Reduced single-cell demultiplexing staff from ~5 to 2

- **(Team Member, 2023. 04 – 2023. 12) AWS cloud lab environment for external training: design and build**

✓ Tech Stack : AWS EC2, CloudFormation, EC2 Spot Instance, Docker, R Studio Server

✓ Key Features

- Designed the training lab environment to be built and reproduced quickly from CloudFormation templates
- Built a cost-efficient, scalable operating structure using Spot Instances
- Improved instance management and environment deployment for both trainees and operators

✓ Result : Built a cost-efficient, scalable cloud operation for training

- **(Project Lead, 2022. 06 – 2022. 08) NGS raw data statistics module improvement and pipeline integration automation**

✓ Tech Stack : Python, Cython, C, Snakemake, Conda, Docker

✓ Key Features

- Analyzed bottlenecks in the existing statistics logic, redesigned the computation structure, and optimized performance
- Validated the prototype in Cython, then implemented the final version in C for high-throughput processing of large NGS data
- Integrated the improved statistics into the analysis pipeline and automated it as a Snakemake workflow (standardized input/output/log/re-run flow)
- Standardized runtime environments with Conda/Docker for easier deployment and reproducibility (environment consistency)

✓ Results

- ~50x speedup; reduced TAT by 3+ days for a 10-sample, 100 GB delivery
- Reduced manual work and improved operational stability/reproducibility through workflow automation and environment standardization

- **(Team Member, 2022. 02 – 2024. 05) NGS analysis pipeline development**

✓ Tech Stack : Bioinformatics tools, Linux, Python, Docker, Snakemake, Conda

✓ Key Features

- Led and contributed to a wide range of pipelines: NGS demultiplexing, single cell (Cell Ranger), WGS, WES, Target-seq, Methyl-seq, PCR raw data analysis, TCR-seq, CRISPR on/off-target, and more
- Supported operation/improvement of modules for each analysis stage and development of data processing automation

✓ Result : Gained broad experience across NGS analysis domains and improved pipeline operational stability

---

Bioinformatics Engineer **Cytogen** **2021. 11 – 2022. 02**

- **(Team Member, 2021. 11 – 2022. 02) TCR-based immune response analysis pipeline: design and operations automation**

✓ Tech Stack : Docker, Snakemake, Python, Linux, Shell, Bioinformatics tools

✓ Key Features

- Turned the TCR immune response analysis methodology into a production-ready pipeline (focused on reproducibility/standardization)
- Containerized the analysis environment with Docker to reduce dependency conflicts and ease deployment/migration
- Automated each workflow stage with Snakemake, with input/output and log management and re-runs from failure points
- Documented Linux/Shell execution scripts and operating procedures, automating repetitive tasks and reducing operational burden

✓ Result : Ensured reproducibility of the analysis process and improved operational stability through execution/operations automation

• **(Team Member, 2021. 11 – 2022. 02) Ion Torrent NGS platform and on-premises operations (DevOps support)**

✓ Tech Stack : Linux, IonTorrent Web Service operations (in-house), Python, Shell

✓ Key Features

- Operated and monitored servers connected to Ion Torrent instruments (service availability/resources)
- Operated and maintained in-house web services installed on the servers (configuration management, incident response, feature improvements)
- Implemented automatic assessment/guidance features in the web service so the lab team could evaluate results quickly

✓ Result : Stabilized instrument–server–web service operations and sped up the lab team's decision-making

---

Bioinformatics Engineer **Labgenomics** **2020. 01 – 2021. 03**

• **(Team Member, 2020. 01 – 2021. 03) Genomic test analysis pipeline operations automation and maintenance**

✓ Tech Stack : Python, Linux, Cron, MySQL

✓ Key Features

- Developed and maintained analysis pipelines for cancer gene panels and pediatric genomic abnormality tests (production environment)
- Automated recurring analyses and scheduled jobs with Cron, improving operational stability and throughput
- Standardized the analysis flow in Python with logging, error handling, and re-run support
- Managed analysis progress and result metadata in MySQL for traceability and operational visibility

✓ Result : Reduced repetitive work and improved pipeline stability through automation and standardization

• **(Team Member, 2020. 04 – 2020. 06) NGS demultiplexing re-analysis (raw data preprocessing) system development and operations automation**

✓ Tech Stack : Python, Linux, Cron, MySQL

✓ Key Features

- Designed and implemented a reprocessing workflow enabling re-analysis of NGS demultiplexing results
- Automated re-analysis requests and scheduled jobs with Cron, improving operational efficiency
- Modularized re-analysis logic in Python with input validation, logging, and error handling for operational stability
- Managed re-analysis requests, progress, and result metadata in MySQL, establishing traceability and history management

✓ Result : Standardized and automated the re-analysis process, shortening lead time and reducing operational burden

Mentoring

• Python Lectures

- Basic syntax

  - Web crawling and web automation
  - Using government public data Open APIs
  - Automated stock trading with the Kiwoom API
  - Data visualization
- GUI program development with PyQt5

Personal Projects

- **(2026. 07 – Present) Korean stock analysis web service (AWS serverless, free public service in operation)**

  ✓ Tech Stack : AWS(Lambda, Cognito, DynamoDB, S3, SES, IAM), React, Vite, Python, Docker, GitHub Pages

  ✓ Key Features

  - Designed a serverless backend of 16 Lambda functions; applied least privilege with per-function IAM policies/roles; managed resource configuration as JSON files and deployed via CLI
  - Multi-layer authentication: SigV4-signed calls (Function URL IAM auth) with Cognito User Pool + Identity Pool temporary credentials, plus JWT (RS256) re-verification inside each Lambda
  - Applied a common gate (authentication · rate limit) at every Lambda entry point; DynamoDB atomic counters automatically disable accounts on detected abuse
  - Frontend on GitHub Pages; data stored in S3 and served directly through Lambda Function URLs (no API Gateway · CloudFront, zero fixed cost); minimized requests with immutable/mutable file separation and a manifest-based incremental cache (IndexedDB)
  - A Docker backend batch uploads only changed data to S3 by comparing file hashes (incremental sync)

  ✓ Results

  - Switched the rate-limit store from S3 to DynamoDB atomic counters, cutting cost from $43 to $5.6 per month at 9M calls (~87% reduction)
  - Kept S3 request cost at about $0.3 per month in normal operation; running as a free public service

- **(2025. 07 – 2025. 10) Video/audio transcription web service (AWS serverless)**

  ✓ Tech Stack : AWS(Cognito, S3, Lambda), React, TypeScript, OpenAI Whisper, CTC aligner, pyannote, ffmpeg, Session Storage, IndexedDB

  ✓ Key Features

  - Built the entire service on an AWS serverless architecture without a database (Cognito-based authentication)
  - Built the transcription pipeline by combining Whisper + CTC aligner + pyannote + ffmpeg with original algorithms
  - Analyzed the Whisper repository code structure to optimize embedding time and apply a model caching strategy
  - Strengthened security with login MFA; reduced AWS request costs using Session Storage/IndexedDB

  ✓ Results

  - Achieved ~99% timestamp accuracy for audio transcription
  - Up to 70% faster processing through model caching/optimization

- **(2024. 01 – 2024. 04) Korean listed-stock database (automated price/financial data collection and normalization)**

  ✓ Tech Stack : MySQL, Redis, Python, Docker, Public Data Portal API, Kiwoom Securities API, FSS Open API

  ✓ Key Features

  - Automatically collected stock prices (daily to yearly bars) and financial statements/indicators for Korean listed companies via APIs
  - Designed schemas and normalized the collected data into a DB suited for querying and analysis
  - Performed analyses on the stock data to derive insights

  ✓ Result : Built an automated pipeline for collecting and normalizing stock price/financial data

- **(2024. 10 – 2025. 02) nand2tetris: Building a computer system from basic logic gates**

✓ Tech Stack : Nand2Tetris HDL, Hack Computer Architecture, Hack Assembly, Hardware Simulator, CPU Emulator, Test Script(.tst/.cmp)

✓ Key Features

- Starting from the NAND gate, designed basic logic gates, combinational circuits (Mux, Adders, etc.), and sequential circuits (registers/memory) in HDL
- Incrementally built a computer architecture including the Hack CPU (registers/ALU/PC) and memory map (ROM/RAM/Screen/Keyboard), verified with emulators
- Wrote Hack Assembly programs (arithmetic, branching, memory access, screen output) to verify "hardware → program execution" end-to-end

✓ Results

- Implemented the gate → CPU → computer hierarchy firsthand, systematically organizing computer architecture and low-level operating principles

✓ github : https://github.com/Shin-jongwhan/nand2tetris

• (2024. 11 – 2024. 12) Personal VPN server for remote work, security, and shared storage (NTFS)

✓ Tech Stack : OpenVPN, Easy-RSA(PKI), Windows Firewall/PowerShell(NAT), Router Port Forwarding, TCP/IP Routing, Windows File Sharing(SMB/NTFS)

✓ Key Features

- Built a VPN server for remote access and configured internal network routing (push route)
- Systematized certificate-based (TLS) client profile (.ovpn) distribution/management
- Documented the "external → internal network" connection from an operations perspective, including firewall rules, port forwarding, and NAT
- Validated remote work scenarios by accessing internal services (DB/file sharing) over the VPN

✓ Result : Built a foundation for stable access to internal resources over a secure tunnel in a personal remote work environment

✓ github : https://github.com/Shin-jongwhan/openvpn

• Personal Docker repositories: 150K+ total pulls

✓ Tech Stack : Docker, Dockerfile, Linux, Container Registry(Docker Hub)

✓ Key Features

- Standardized analysis/service runtime environments as container images to minimize "dev ↔ prod" differences (reproducibility/portability)
- Consistently managed image releases with a tag/version policy and built image-based deployment flows for pipelines/workflows
- Built common base images and tool-specific images to speed up environment setup/updates, reused in Kubernetes service operations

✓ Result : 150K+ cumulative pulls on personal Docker Hub; continuously building experience in container-centric deployment and environment standardization

✓ dockerhub : https://hub.docker.com/u/shinejh0528

Technical Reading

• Deep learning from scratch 3

- Keywords : How modern deep learning frameworks work, building a deep learning framework from scratch (differentiation, forward, backward), object-oriented programming centered on Python classes, incremental and systematic software development, computational graph generation via Define-by-Run

• Introduction to Natural Language Processing with Deep Learning

- Keywords : Natural language processing, text preprocessing (embedding), statistical language models, n-gram, RNN, LSTM, GRU, word2vec, GloVe

• Deep learning from scratch 2

- Keywords : Natural language processing, thesaurus, count-based NLP, word2vec, CBOW, inference-based NLP, RNN, LSTM, GRU, seq2seq, attention

• The Elements of Computing Systems (Nand2Tetris) - From Boolean logic to computer architecture and operating systems

- Keywords : Boolean logic, logic gates, Boolean/binary arithmetic, memory, flip-flops, machine language, Hack machine language, HDL, von Neumann architecture, assembler, high-level languages, compiler

• Deep learning from scratch

- Read 3 times

- Keywords : Machine learning basics, perceptrons, neural networks, linear and nonlinear functions, sigmoid, ReLU, softmax, Affine layer, Softmax-with-Loss layer, handwritten digit (0–9) image inference model, cross-entropy error, differentiation, backpropagation, numerical differentiation and gradient descent, parameter update methods (Momentum, AdaGrad, Adam), weight initialization, CNN

• High performance python

- Keywords : Compiling Python to C (Cython), multiprocessing, multithreading, time/space complexity optimization

• IT Infrastructure Explained Through Illustrations

- Keywords : Vertical/horizontal/geographic partitioning architectures, computer hardware components, service infrastructure design, networking, 3-tier architecture, infrastructure fundamentals (sync/async, serial/parallel, queues, stateful/stateless), service redundancy

• Introduction to Machine Learning with Python

- Keywords : Traditional machine learning with scikit-learn, supervised learning, classification and regression, KNN, linear models, naive Bayes, decision trees, kernel SVM, unsupervised learning, dimensionality reduction (PCA, t-SNE), clustering (k-means, agglomerative), count-based NLP practice (CBOW, n-gram)

Developer Communities

• Nextflow workflow language KR (Nextflow user group / hosted invited-speaker talks, 2023. 02 ~)
• AWSKRUG (AWS Korea User Group / 2025. 02 ~ 2025. 12)
• Online study group (Cloud, DevOps / organizer / 2025.01 ~ 2025.07)

Interests

• **Investing** : Stocks, real estate
• **Books & Philosophy** : Favorite philosopher: Kant. Most memorable book: Arthur C. Clarke's Childhood's End.
• **Hobby** : Singing

About Me

Hello, I'm Jonghwan Shin, aiming to grow as a DevOps and Software Engineer with strengths in Python backend development and building large-scale analysis automation systems. Through diverse projects, I have worked across a broad range of technical areas, including backend, data processing, cloud infrastructure, high-availability architecture design, and automation systems, while focusing on solving real-world problems with technology. Majoring in life science, I naturally developed an interest in data analysis and computing environments through bioinformatics. In industry, I have gained hands-on, production-ready skills including developing 10+ NGS analysis pipelines, analysis automation, distributed processing on server clusters, MySQL replication, and disaster recovery automation. My core strengths are automating analysis pipelines to reduce operational risk and efficiently managing petabyte-scale data while delivering it as reliable services. Beyond simply using technologies, I have consistently thought about how to design and connect systems, and what it takes to offer them as services anyone can use. Operating services on Kubernetes, parallel processing on large server clusters, Snakemake automation, CI/CD pipelines, high-availability database configurations, and IaC tooling all extend from that mindset.
To deepen my expertise in IT infrastructure and cloud, I earned three AWS certifications as a way to design and validate the

distributed systems and cloud architectures I operate. By providing external training environments with AWS EC2, RDS, and CloudFormation and through personal projects, I connected theory with practice and gained confidence that my skills apply broadly across domains.

In each project, I have built depth by understanding the domain and selecting and implementing the right technologies. Believing that technology is always a tool for solving problems, every technology I have used had a clear rationale and design purpose. Going forward, I aim to be an engineer whom teams and organizations can trust, grounded in technical reliability and operational stability. Thank you.